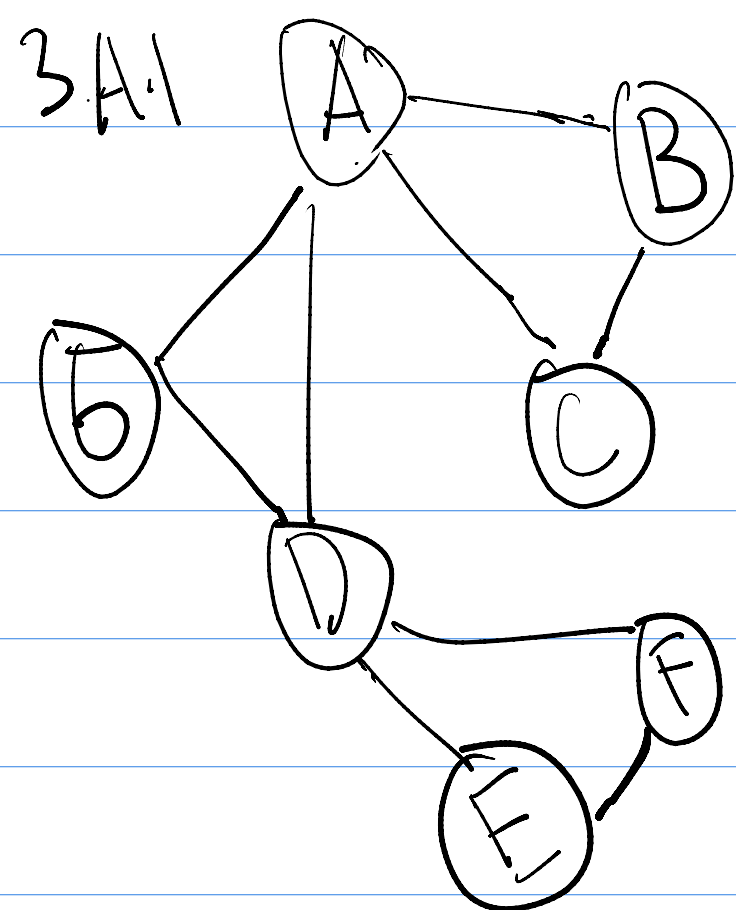
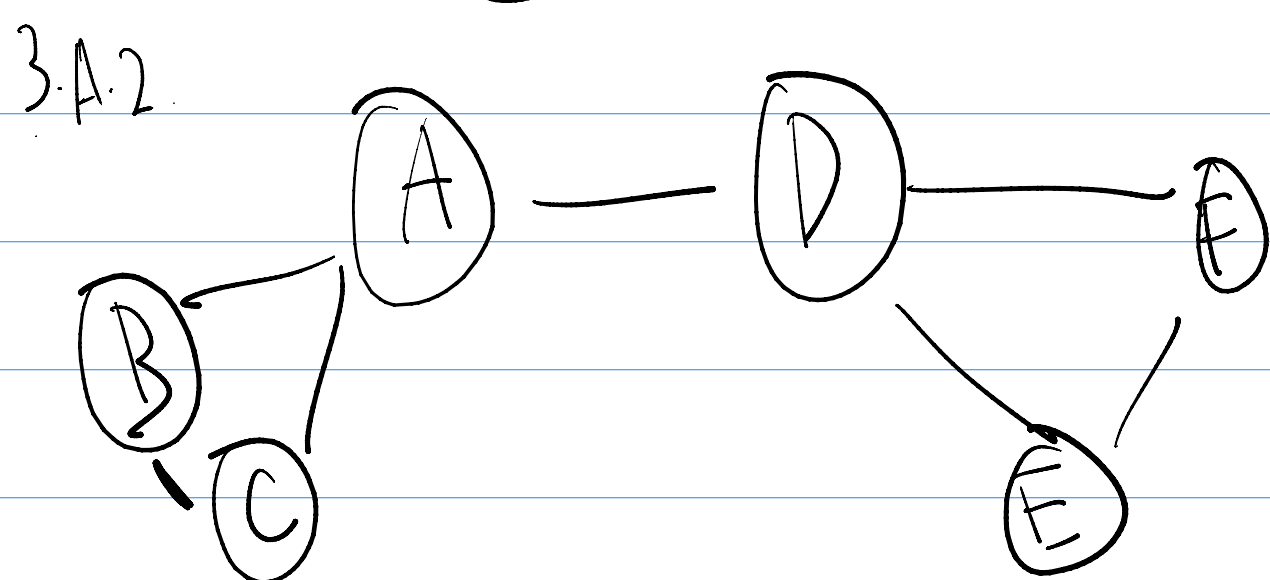


# Network science - Ex 03



1. Open Triads =  $(B, A, D)$ ,  $(C, A, D)$ ,  $(E, D, A)$ ,  $(F, D, A)$
2. Each missing edge is supported by exactly 1 open triad  
So, they have similar closure pressure
3. Either  $(B, D)$  or  $(C, D)$  they be
4.  $(C, E)$ ,  $(F, E)$



## Local clustering coefficient

1.  $C(B)$ ,  $C(C)$ ,  $C(E)$ ,  $C(F) = 1$ , since all of their neighbors are connected  
 $C(A) = \frac{1}{3}$ , B, C, D | only B-C connected  
 $C(D) = \frac{1}{3}$ , E, F, A | only E-F connected

## 2. average clustering coefficient

$$\bar{C} = (1 + 1 + 1 + 1 + \frac{1}{3} + \frac{1}{3}) / 6 = \frac{14}{10} = \frac{7}{5}$$

3.  $\bar{C}$  of 0.70 is considered 'high' which means Friends of friends are more likely to be friend

- 3.B.1
1. Yes removing either bridges destroy the cliques
  2. Yes, node 3-4 has no common neighbor beside each other

## Local Bridge

an edge whose endpoints share no other neighbor

## Structural holes

Gaps between otherwise disconnected parts of a social network

3. Yes. Since Clique 3 and 1 have no direct connection. Node 5 via (4 and 6)  
Can broker informations through these disjoint worlds
4. the structural hole closes. edge 6-7 is no longer a bridge since edge 3-7 now exists

## 3.B.2

```
study > snippets > network-ex03 > 3.B.2.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3
4 G = nx.Graph()
5 G.add_edges_from([(1,2),(1,3),(2,3),(4,5),(4,6),(5,6),(7,8),(7,9),(8,9),(3,4),(6,7)])
6 bridges = list(nx.bridges(G))
7 # [(3,4), (6,7)]
8 ebc = nx.edge_betweenness centrality(G)
9
10 print("Bridges:", bridges)
11 sorted_eb = sorted(ebc.items(), key=lambda kv: kv[1], reverse=True)
12 print("Edges ranked by edge betweenness:")
13 for edge, score in sorted_eb:
14     print(f"{edge}: {score:.3f}")
15
16 plt.figure(figsize=(8, 6))
17 pos = nx.spring_layout(G, seed=42)
18 # safe edge-width lookup (undirected edges may appear in either order)
```

```

19 edge_widths = [ebc.get(tuple(edge), ebc.get((edge[1], edge[0]), 0)) * 5 for edge in G.edges()]
20 # draw nodes and edges scaled by betweenness
21 nx.draw(G, pos, with_labels=True, width=edge_widths, node_color='lightblue', edgecolors='gray')
22 # draw bridges on top in red
23 nx.draw_networkx_edges(G, pos, edgelist=bridges, width=4, edge_color='red')
24
25 import matplotlib.lines as mlines
26 bridge_line = mlines.Line2D([], [], color='red', linewidth=4, label='Bridges')
27 plt.legend(handles=[bridge_line])
28 plt.title('Edge Betweenness Centrality (bridges highlighted)')
29 plt.show()

```

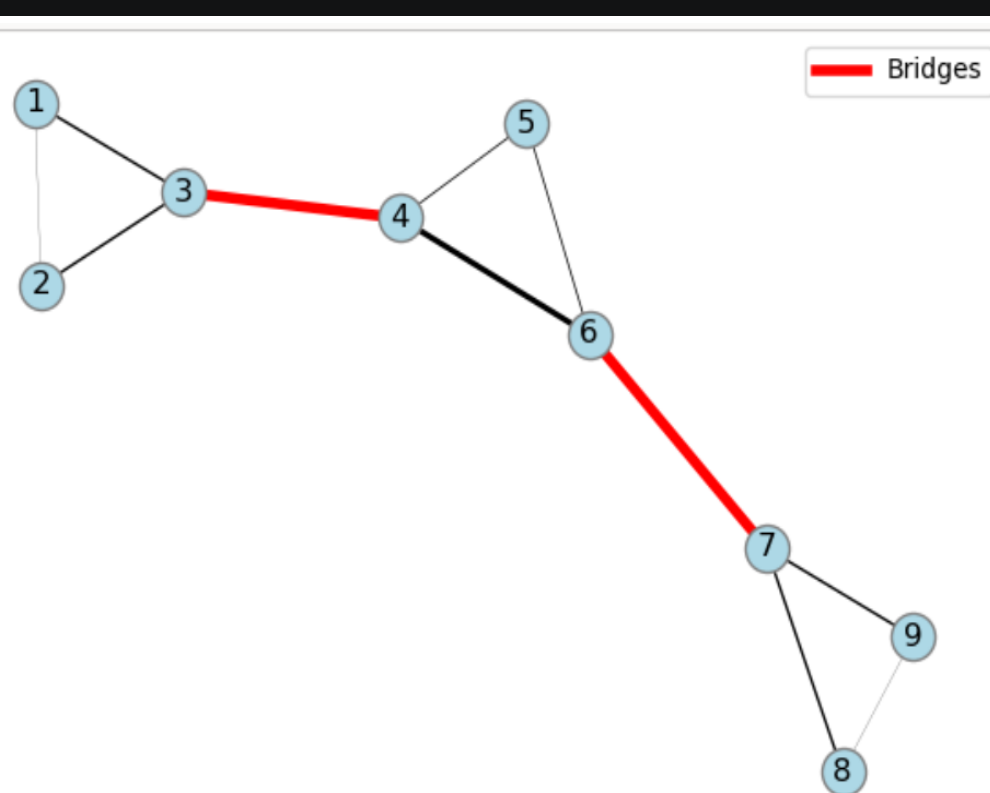
PROBLEMS OUTPUT DEBUG CONSOLE

Edges ranked by edge betweenness:

```

(3, 4): 0.500
(6, 7): 0.500
(4, 6): 0.444
(1, 3): 0.194
(2, 3): 0.194
(7, 8): 0.194
(7, 9): 0.194
(4, 5): 0.111
(5, 6): 0.111
(1, 2): 0.028
(8, 9): 0.028

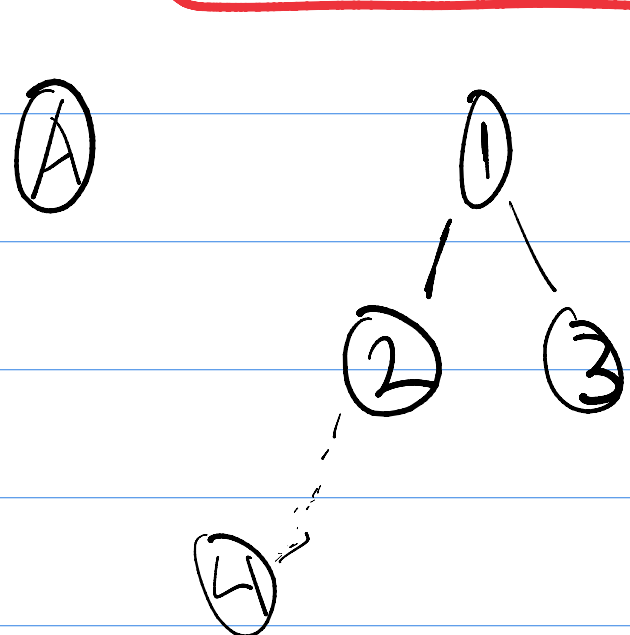
```



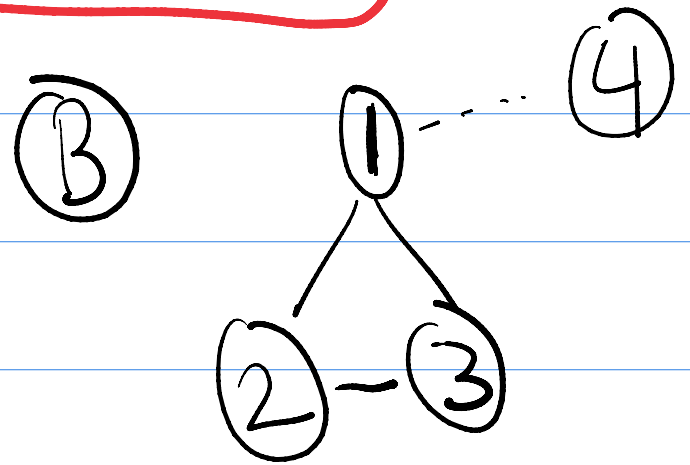
output

### 3.C.1 Weak Ties Theorem

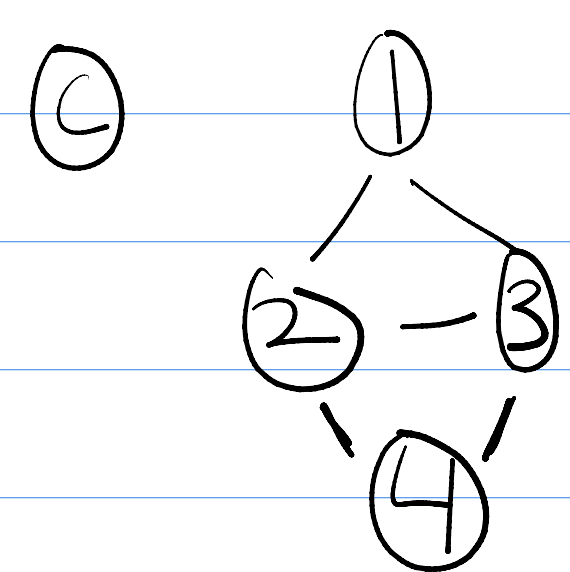
Under Strong Triadic Closure every local bridge is a weak tie



STC Failed. 1 is connected to 2 and 3 but 2-3 doesn't form



STC Satisfied. 1-4 is a bridge and is weak

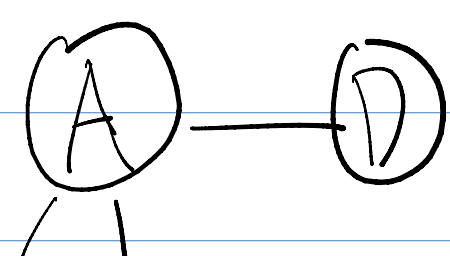


STC Failed 2 and 3 is tied to 1 and 4 but there's no 1-4 edge.

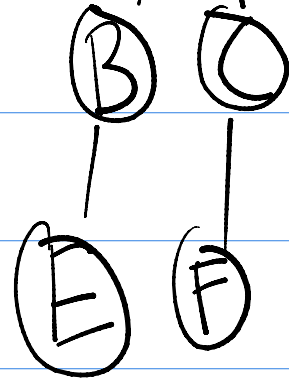
- ### 3.C.2
1. Because weak ties are more likely to connect us to other networks rather than close friends that usually have the same clusters of friends
  2. Strong ties cluster in dense groups which means redundancy in information, weak ties bridges to different clusters (novel information)
  3. No, strong friends provide trust and support, weak ties provide reach. We need both
  4. Formalises weak ties with low neighbourhood overlap, enabling people to seek their reach into structural holes

### 3.D.1 Neighbourhood overlap

$|N(u) \cap N(v)|$



$$O(u,v) = \frac{|N(u) \cap N(v)|}{|N(u) \cup N(v)|}$$



$$① \quad O(A,B) = \frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|} = \frac{1}{3}$$

$$O(A,C) = \frac{|N(A) \cap N(C)|}{|N(A) \cup N(C)|} = \frac{1}{3}$$

$$O(B,E) = \frac{\emptyset}{|N(A) \cup N(B)|} = 0$$

②  $O(B,E)$  has zero overlap, which is perfect as a local bridge

3.D.2

```
study > snippets > network-ex03 > 3.D.2.py > ...
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import numpy as np
4 + import matplotlib.lines as mlines  ↳ Tab to jump here · Esc to dis
5 G = nx.karate_club_graph()
6
7 def neighbour_overlap(G, u, v):
8     """Compute the neighbour overlap between nodes u and v in graph G."""
9     neighbors_u = set(G.neighbors(u))
10    neighbors_v = set(G.neighbors(v))
11    intersection = len(neighbors_u & neighbors_v)
12    union = len(neighbors_u | neighbors_v)
13    if union == 0:
14        return 0
15    return intersection / union
16
17 #compare within and across factions
18
19 within = [neighbour_overlap(G, u, v) for u, v in G.edges()
20           if G.nodes[u]['club'] == G.nodes[v]['club']]
21 across = [neighbour_overlap(G, u, v) for u, v in G.edges()
22           if G.nodes[u]['club'] != G.nodes[v]['club']]
23
24 print(f"Within-faction overlap: {np.mean(within):.3f}")
25 print(f"Across-faction overlap: {np.mean(across):.3f}")
26
```

```
> python3 3.D.2.py
Within-faction overlap: 0.153
Across-faction overlap: 0.059
```

3.E.10 Symmetric (undirected edges) = Facebook friends

Asymmetric (directed edges) = Twitter Follow

Twitter Follows are closer to structural bridges with near-zero overlap

② Maintained relationship grows quickly because it requires time, emotional labor which means they hit cognitive limit "Faster"

while total count (degree, followers, connections) grow quicker because it is easy (one click to add/follow), low-trust, and weak-ties connects to further reach

3.E.2

```
study > snippets > network-ex03 > 3.E.2.py > topological_overlap
1 import networkx as nx
2 import matplotlib.pyplot as plt
3 import numpy as np
4
5 G = nx.karate_club_graph()
6
```

```

7 strengths, overlaps = [], []
8
9 #topological overlap function (accounts for the edge itself)
10 def topological_overlap(G, u, v):
11     """Topological overlap  $T(u,v) = (|N(u) \cap N(v)| + 1) / \min(\deg(u), \deg(v))$  for adjacent  $u,v$ ."""
12     neighbors_u = set(G.neighbors(u))
13     neighbors_v = set(G.neighbors(v))
14     intersection = len(neighbors_u & neighbors_v)
15     denom = min(len(neighbors_u), len(neighbors_v))
16     return (intersection + 1) / denom if denom > 0 else 0
17
18 #Compute strengths and overlaps
19 for u, v in G.edges():
20     s = 0.8 if G.nodes[u]['club'] == G.nodes[v]['club'] else 0.2
21     strengths.append(s)
22     overlaps.append(topological_overlap(G, u, v))
23
24
25 #Correlation
26 r = np.corrcoef(strengths, overlaps)[0, 1]
27 print(f"Pearson correlation coefficient: {r:.3f}")

```

```

> python3 3.E.2.py
Pearson correlation coefficient: 0.573

```

Pearson correlation is typically  $> 0.6$ . Confirming the tie strength and neighbourhood overlap are linked: close-knit groups share friend, while bridging ties remain socially isolated.